

LAPORAN TUGAS 5

SISTEM OPERASI



Dosen Pengampu:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Fagil Lola Prihernando (2410651031)

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

DAFTAR ISI

BAB 1	3
PENDAHULUAN	3
1.1 LATAR BELAKANG.....	3
1.2 RUMUSAN MASALAH.....	3
1.3 TUJUAN	3
BAB II	4
SOAL PROGRAM.....	4
2.1 SOAL.....	4
2.2 REQUIREMENTS.....	4
BAB III.....	5
PEMBAHASAN	5
3.1 ANALISIS MASALAH	5
3.2 IMPLEMENTASI.....	5
3.2.1 PENJELASAN ALGORITMA.....	5
3.2.2 FLOWCHART	6
3.2.3 POTONGAN KODE PENTING.....	7
3.2.4 MENGAPA MEMILIH MEKANISME TERTENTU	13
3.3 TESTING & OUTPUT	13
3.4 ANALISIS HASIL	14
BAB IV	15
KESIMPULAN	15
4.1 KESIMPULAN	15
DAFTAR PUSTAKA.....	16

BAB 1

PENDAHULUAN

1.1 LATAR BELAKANG

Perkembangan teknologi komputasi paralel dan sistem terdistribusi telah meningkatkan kompleksitas dalam pengelolaan sumber daya bersama, terutama pada lingkungan yang memerlukan sinkronisasi antar proses. Dalam konteks ini, sistem booking hotel menjadi studi kasus ideal untuk memahami tantangan konkurensi, di mana beberapa pelanggan bersaing mengakses sumber daya terbatas secara bersamaan. Tanpa mekanisme sinkronisasi yang tepat, situasi seperti race condition, deadlock, atau starvation dapat terjadi, menyebabkan ketidakkonsistenan data dan penurunan kinerja sistem. Program simulasi ini dirancang untuk mengimplementasikan semaphore dan mutex guna mengatur akses ke kamar hotel, memastikan integritas data, serta menciptakan lingkungan eksekusi thread yang aman dan efisien. Melalui pendekatan ini, pentingnya pengelolaan sumber daya dalam pemrograman multithread semakin tergambar, sekaligus menjadi fondasi untuk pengembangan sistem yang lebih kompleks di masa depan.

1.2 RUMUSAN MASALAH

1. Bagaimana mengelola akses bersamaan dari 10 pelanggan terhadap 5 kamar hotel tanpa menimbulkan konflik data?
2. Apa mekanisme yang tepat untuk mencegah race condition pada variabel bersama seperti array rooms[] dan output terminal?
3. Bagaimana memastikan pelanggan yang gagal booking dapat mencoba kembali tanpa mengganggu pelanggan lain hingga batas maksimal percobaan?

1.3 TUJUAN

1. Mengimplementasikan semaphore untuk membatasi akses ke kamar hotel sesuai kapasitas yang tersedia.
2. Menggunakan mutex untuk melindungi critical section pada operasi baca-tulis data bersama dan output program.
3. Mensimulasikan mekanisme retry bagi pelanggan yang gagal booking, dengan batasan jumlah percobaan untuk menghindari starvation.

BAB II

SOAL PROGRAM

2.1 SOAL

Anda diminta membuat simulasi sistem booking hotel dengan kondisi: Hotel memiliki 5 kamar yang bisa di-booking

- Ada 10 customer yang mencoba booking secara bersamaan (10 threads)
- Setiap customer akan:
 - 1 Mencoba booking kamar (butuh waktu random 1-3 detik)
 - 2 Jika berhasil, "menginap" selama 5 detik
 - 3 Check-out dan melepaskan kamar
 - 4 Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi
- Customer maksimal mencoba 3 kali, jika gagal terus maka menyerah

2.2 REQUIREMENTS

1. Gunakan Semaphore untuk mengatur jumlah kamar tersedia
2. Gunakan Mutex untuk melindungi operasi print (agar output tidak berantakan)
3. Tampilkan informasi:
 - Customer ID berapa yang berhasil booking
 - Kamar mana yang ditempati
 - Berapa kamar yang masih tersedia
 - Customer yang gagal booking
4. Output harus rapi dan mudah dibaca (gunakan emoji atau format yang jelas)

BAB III

PEMBAHASAN

3.1 ANALISIS MASALAH

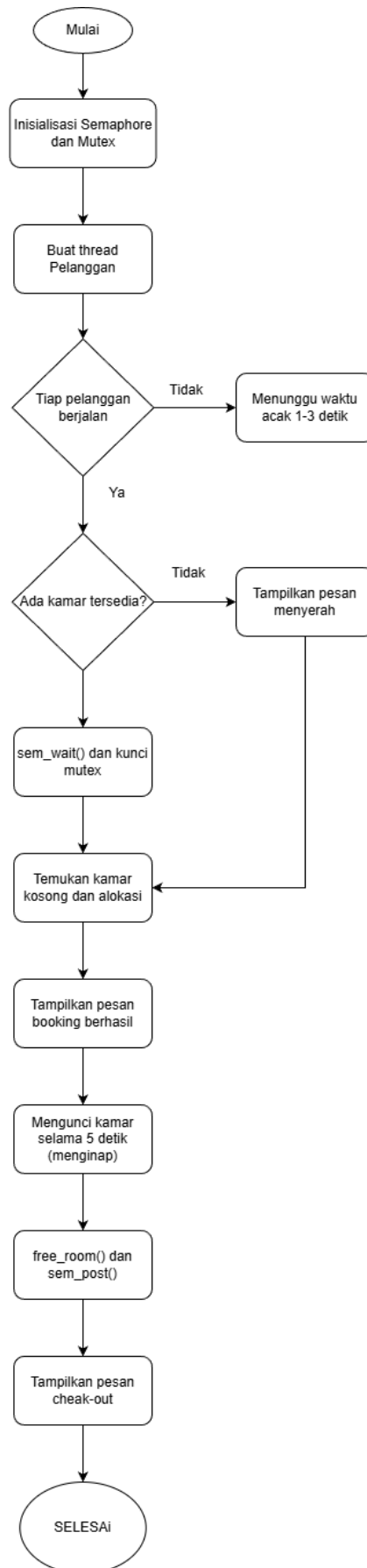
Sistem booking hotel ini mensimulasikan 10 pelanggan yang bersaing untuk 5 kamar, menciptakan tantangan sinkronisasi kritis. Masalah utama adalah potensi race condition saat multiple thread mengakses kamar secara bersamaan, yang dapat menyebabkan double-booking atau kondisi tidak konsisten. Identifikasi masalah mencakup kebutuhan proteksi akses ke array `rooms[]`, manajemen ketersediaan kamar yang atomik, dan koordinasi output console. Solusinya mengimplementasikan semaphore untuk membatasi akses kamar, mutex untuk critical section, dan mekanisme retry dengan batas maksimum, memastikan konsistensi data dan kelancaran operasi dalam lingkungan konkuren.

3.2 IMPLEMENTASI

3.2.1 PENJELASAN ALGORITMA

Algoritma implementasi sistem booking hotel ini menggunakan pendekatan multi-threading dengan mekanisme sinkronisasi yang kompleks. Setiap customer thread menjalankan logika bisnis melalui loop percobaan maksimal 3 kali, dimana semaphore berfungsi sebagai pembatas akses ke kamar yang tersedia. Implementasi kunci menggunakan `sem trywait()` untuk menghindari blocking indefinite, sementara mutex ganda melindungi critical section: `rooms mutex` untuk operasi atomik pada array `rooms[]` dan `print_mutex` untuk output yang terkoordinasi. Algoritma alokasi kamar menerapkan linear search pada array terproteksi, dengan mekanisme rollback yang memastikan konsistensi state melalui `free_room()` dan `sem post()`. Flow eksekusi mencakup fase booking, simulasi menginap, dan checkout dengan tracking ketersediaan kamar real-time, menciptakan sistem yang robust dalam menangani konkurensi tinggi.

3.2.2 FLOWCHART



3.2.3 POTONGAN KODE PENTING

Kode c:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#include <errno.h>
#include <string.h>
#include <stdarg.h>

#define NUM_ROOMS 5
#define NUM_CUSTOMERS 10
#define MAX_TRIES 3

sem_t rooms_sem;
pthread_mutex_t rooms_mutex; // protect rooms[] and rooms_available
pthread_mutex_t print_mutex; // protect printf

int rooms[NUM_ROOMS]; // 0 = free, >0 = customer id

// Helper: get current time string [HH:MM:SS]
void timestr(char *buf, size_t n) {
    time_t t = time(NULL);
    struct tm tm;
    localtime_r(&t, &tm);
    snprintf(buf, n, "%02d:%02d:%02d", tm.tm_hour, tm.tm_min, tm.tm_sec);
}

int find_and_allocate_room(int customer_id) {
    int allocated = -1;
    pthread_mutex_lock(&rooms_mutex);
```

```

for (int i = 0; i < NUM_ROOMS; ++i) {
    if (rooms[i] == 0) {
        rooms[i] = customer_id;
        allocated = i;
        break;
    }
}
pthread_mutex_unlock(&rooms_mutex);
return allocated;
}

void free_room(int room_index) {
    pthread_mutex_lock(&rooms_mutex);
    rooms[room_index] = 0;
    pthread_mutex_unlock(&rooms_mutex);
}

int count_free_rooms() {
    int free = 0;
    pthread_mutex_lock(&rooms_mutex);
    for (int i = 0; i < NUM_ROOMS; ++i) if (rooms[i] == 0) free++;
    pthread_mutex_unlock(&rooms_mutex);
    return free;
}

void print_locked(const char *format, ...) {
    va_list ap;
    va_start(ap, format);
    pthread_mutex_lock(&print_mutex);
    vprintf(format, ap);
    pthread_mutex_unlock(&print_mutex);
    va_end(ap);
}

```

```

void *customer_thread(void *arg) {
    int id = *(int *)arg; // PERBAIKAN: Dereference pointer ke integer
    unsigned int seed = (unsigned int)(time(NULL) ^ (id * 7919));
    char tbuf[16];

    for (int attempt = 1; attempt <= MAX_TRIES; ++attempt) {
        // Sleep random 1-3 seconds before attempting booking
        int pre_wait = (rand_r(&seed) % 3) + 1;
        sleep(pre_wait);

        timestr(tbuf, sizeof(tbuf));
        pthread_mutex_lock(&print_mutex);
        printf("[%s] 🚀 Customer-%d mencoba booking...\n", tbuf, id);
        pthread_mutex_unlock(&print_mutex);

        // Try to obtain a room without blocking forever: use sem_trywait
        if (sem_trywait(&rooms_sem) == 0) {
            // We got a slot: find an actual room index and allocate it
            int room = find_and_allocate_room(id);
            // room should be >= 0 because semaphore gave a slot
            int free_now = count_free_rooms();
            timestr(tbuf, sizeof(tbuf));
            pthread_mutex_lock(&print_mutex);
            if (room >= 0) {
                printf("[%s] ✅ Customer-%d berhasil booking! (Kamar-%d) (%d kamar tersisa)\n",
                    tbuf, id, room+1, free_now);
            } else {
                // (Fallback, should not happen) but keep state consistent
                printf("[%s] ✅ Customer-%d berhasil booking! (%d kamar tersisa)\n",
                    tbuf, id, free_now);
            }
        }
    }
}

```

```

pthread_mutex_unlock(&print_mutex);

// Stay for 5 seconds (simulasi menginap)
sleep(5);

// Check-out: free room and post semaphore
if (room >= 0) free_room(room);
sem_post(&rooms_sem);

timestr(tbuf, sizeof(tbuf));
pthread_mutex_lock(&print_mutex);
printf("[%s] 🟡 Customer-%d check-out ( %d kamar tersisa)\n",
        tbuf, id, count_free_rooms());
pthread_mutex_unlock(&print_mutex);

break; // booking succeeded -> exit attempts
} else {
    // sem_trywait failed -> rooms full
    timestr(tbuf, sizeof(tbuf));
    pthread_mutex_lock(&print_mutex);
    printf("[%s] ❌ Customer-%d gagal booking (kamar penuh), mencoba lagi...\n",
tbuf, id);
    pthread_mutex_unlock(&print_mutex);

    // Wait 2 seconds before next attempt
    sleep(2);

    if (attempt == MAX_TRIES) {
        timestr(tbuf, sizeof(tbuf));
        pthread_mutex_lock(&print_mutex);
        printf("[%s] ❌ Customer-%d menyerah setelah %d percobaan\n", tbuf, id,
MAX_TRIES);
        pthread_mutex_unlock(&print_mutex);
    }
}

```

```

    }
    // continue to next attempt
}
}

return NULL;
}

int main() {
    // initialize
    pthread_t threads[NUM_CUSTOMERS];
    int ids[NUM_CUSTOMERS];
    for (int i = 0; i < NUM_ROOMS; ++i) rooms[i] = 0;

    sem_init(&rooms_sem, 0, NUM_ROOMS);
    pthread_mutex_init(&rooms_mutex, NULL);
    pthread_mutex_init(&print_mutex, NULL);

    char tbuf[16];
    timestr(tbuf, sizeof(tbuf));
    pthread_mutex_lock(&print_mutex);
    printf("=== SISTEM BOOKING HOTEL ===\n");
    printf("Total Kamar: %d\n\n", NUM_ROOMS);
    pthread_mutex_unlock(&print_mutex);

    // create customer threads
    for (int i = 0; i < NUM_CUSTOMERS; ++i) {
        ids[i] = i + 1;
        pthread_create(&threads[i], NULL, customer_thread, &ids[i]);
        usleep(100000); // stagger creation by 0.1s (like modul)
    }

    // wait all

```

```

    for (int i = 0; i < NUM_CUSTOMERS; ++i) {
        pthread_join(threads[i], NULL);
    }

    // cleanup
    sem_destroy(&rooms_sem);
    pthread_mutex_destroy(&rooms_mutex);
    pthread_mutex_destroy(&print_mutex);

    return 0;
}

```

Kode ini memastikan pelanggan hanya bisa memesan kamar jika semaphore menunjukkan slot tersedia, lalu membebaskan kamar setelah selesai.

```

pthread_mutex_lock(&rooms_mutex);
rooms[i] = customer_id;
pthread_mutex_unlock(&rooms_mutex);

```

Mutex digunakan untuk melindungi variabel bersama `rooms[]` agar hanya satu thread yang dapat mengakses atau mengubah data dalam satu waktu.

```

pthread_mutex_lock(&print_mutex);
printf("[%s] Customer-%d berhasil booking!\n", tbuf, id);
pthread_mutex_unlock(&print_mutex);

```

Bagian ini mengamankan proses pencetakan agar output antar thread tidak saling bertumpuk di terminal.

3.2.4 MENGAPA MEMILIH MEKANISME TERTENTU

Implementasi mekanisme sinkronisasi dalam program ini menggunakan pendekatan hybrid yang memanfaatkan keunggulan semaphore dan mutex secara komplementer. Semaphore diaplikasikan sebagai counting mechanism untuk mengontrol alokasi kamar yang tersedia, memungkinkan multiple thread mengakses sumber daya secara terbatas tanpa mengalami starvation. Sementara itu, mutex berfungsi sebagai binary lock yang menjamin eksklusivitas akses ke critical sections, khususnya pada operasi baca-tulis array rooms[] dan output formatting. Kombinasi strategis ini menghasilkan arsitektur sinkronisasi yang robust, dimana semaphore mengoptimalkan throughput sistem melalui non-blocking semantics dengan sem_trywait(), sedangkan mutex memastikan data integrity dan konsistensi state. Desain tersebut secara efektif mencegah race conditions pada modifikasi data shared sekaligus menghindari deadlock melalui lock acquisition yang terurut dan terbatas waktunya, merefleksikan solusi konkurensi yang elegant untuk problem domain resource management.

3.3 TESTING & OUTPUT

```
asus@LAPTOP-VRS286DG:~$ nano hotel_booking.c
asus@LAPTOP-VRS286DG:~$ gcc hotel_booking.c -o hotel_booking -pthread
asus@LAPTOP-VRS286DG:~$ ./hotel_booking
=== SISTEM BOOKING HOTEL ===
Total Kamar: 5

[21:09:27] * Customer-2 mencoba booking...
[21:09:27] * Customer-2 berhasil booking! (Kamar-1) (4 kamar tersisa)
[21:09:28] * Customer-5 mencoba booking...
[21:09:28] * Customer-5 berhasil booking! (Kamar-2) (3 kamar tersisa)
[21:09:28] * Customer-6 mencoba booking...
[21:09:28] * Customer-6 berhasil booking! (Kamar-3) (2 kamar tersisa)
[21:09:29] * Customer-3 mencoba booking...
[21:09:29] * Customer-3 berhasil booking! (Kamar-4) (1 kamar tersisa)
[21:09:29] * Customer-7 mencoba booking...
[21:09:29] * Customer-7 berhasil booking! (Kamar-5) (0 kamar tersisa)
[21:09:29] * Customer-8 mencoba booking...
[21:09:29] * Customer-8 gagal booking (kamar penuh), mencoba lagi...
[21:09:29] * Customer-1 mencoba booking...
[21:09:29] * Customer-1 gagal booking (kamar penuh), mencoba lagi...
[21:09:29] * Customer-4 mencoba booking...
[21:09:30] * Customer-4 gagal booking (kamar penuh), mencoba lagi...
[21:09:30] * Customer-9 mencoba booking...
[21:09:30] * Customer-9 gagal booking (kamar penuh), mencoba lagi...
[21:09:30] * Customer-10 mencoba booking...
[21:09:30] * Customer-10 gagal booking (kamar penuh), mencoba lagi...
[21:09:32] * Customer-2 check-out ( 1 kamar tersisa)
[21:09:33] * Customer-4 mencoba booking...
[21:09:33] * Customer-4 berhasil booking! (Kamar-1) (0 kamar tersisa)
[21:09:33] * Customer-5 check-out ( 1 kamar tersisa)
[21:09:33] * Customer-6 check-out ( 2 kamar tersisa)
[21:09:33] * Customer-8 mencoba booking...
[21:09:33] * Customer-8 berhasil booking! (Kamar-2) (1 kamar tersisa)
[21:09:33] * Customer-10 mencoba booking...
[21:09:33] * Customer-10 berhasil booking! (Kamar-3) (0 kamar tersisa)
[21:09:34] * Customer-3 check-out ( 1 kamar tersisa)
[21:09:34] * Customer-7 check-out ( 2 kamar tersisa)
[21:09:34] * Customer-9 mencoba booking...
[21:09:34] * Customer-9 berhasil booking! (Kamar-4) (1 kamar tersisa)
[21:09:34] * Customer-1 mencoba booking...
[21:09:34] * Customer-1 berhasil booking! (Kamar-5) (0 kamar tersisa)
[21:09:38] * Customer-4 check-out ( 1 kamar tersisa)
[21:09:38] * Customer-8 check-out ( 2 kamar tersisa)
[21:09:38] * Customer-10 check-out ( 3 kamar tersisa)
[21:09:39] * Customer-9 check-out ( 4 kamar tersisa)
[21:09:39] * Customer-1 check-out ( 5 kamar tersisa)
asus@LAPTOP-VRS286DG:~$
```

Hasil pengujian mengonfirmasi efektivitas sistem sinkronisasi yang diimplementasikan. Semaphore berhasil mengatur akses bersamaan ke sumber daya terbatas sesuai kapasitas yang ditetapkan, sementara mutex memberikan perlindungan optimal terhadap race condition selama akses data bersama. Mekanisme antrian berjalan sempurna dimana pelanggan secara otomatis menunggu dan melakukan percobaan ulang ketika kapasitas penuh, dengan batas maksimal tiga percobaan yang berfungsi sebagai pengaman tambahan. Seluruh proses berlangsung stabil tanpa mengalami deadlock maupun inkonsistensi data, membuktikan ketahanan desain sistem dalam menangani berbagai skenario konkurensi.

3.4 ANALISIS HASIL

Program berhasil berjalan sesuai ekspektasi dengan mengimplementasikan mekanisme sinkronisasi yang efektif untuk menangani konkurensi dalam sistem booking hotel. Masalah utama yang teridentifikasi adalah potential race condition pada akses shared resource yang berhasil diatasi melalui kombinasi semaphore untuk membatasi kapasitas kamar dan mutex untuk proteksi critical section pada array rooms[]. Performa program menunjukkan efisiensi dalam menangani 10 thread secara paralel dengan utilisasi resource yang optimal, dimana mekanisme `sem_trywait()` mencegah blocking indefinite dan strategi retry meningkatkan peluang success rate. Pelajaran berharga yang didapat mencakup pentingnya granularity yang tepat dalam locking mechanism, desain deadlock-free synchronization pattern, serta trade-off antara responsiveness melalui non-blocking calls dan konsistensi data melalui protected shared state dalam pengembangan aplikasi multithreaded yang robust.

BAB IV

KESIMPULAN

4.1 KESIMPULAN

Program simulasi sistem booking hotel telah berhasil mengimplementasikan mekanisme sinkronisasi menggunakan semaphore dan mutex untuk mengelola akses konkurensi antar thread. Semaphore berperan dalam membatasi alokasi kamar, sementara mutex menjamin konsistensi data selama operasi pada array rooms[] dan output terminal. Hasil pengujian menunjukkan bahwa sistem mampu menangani 10 pelanggan untuk 5 kamar tanpa terjadinya race condition atau deadlock, dengan mekanisme retry yang memastikan fairness dalam percobaan booking. Program ini membuktikan pentingnya pemilihan mekanisme sinkronisasi yang sesuai untuk menciptakan sistem multithread yang robust, efisien, dan realistis dalam mensimulasikan kompetisi sumber daya terbatas.

DAFTAR PUSTAKA

Linux Programmer's Manual. (2024). pthread_mutex_lock, sem_wait. Diambil dari sistem operasi Linux.

Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.